

1. Média do Aluno

Entrada: nota (nota)

Processamento:

- armazenamento das 5 notas em uma lista
- soma das notas
- cálculo da média
- verificação da situação (aprovado, recuperação, reprovado)

Saída: apresentar a situação do aluno

Algoritmo

1. Declarar e inicializar as variáveis `notas = [ ]` como lista, `avg = 0.0` e `nota = 0.0` do tipo real.
2. Para `count` no intervalo de 1 até 5, executar:
 - i. Tentar ler um valor real e armazenar em `nota`.
 - ii. Adicionar `nota` ao final da lista `notas`.
 - iii. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
3. Calcular a média: `avg = sum(notas) / len(notas)`.
4. Apresentar uma linha separadora.
5. Se `avg` for maior ou igual a 7, apresentar mensagem de aprovação.
6. Senão, se `avg` for maior ou igual a 5, apresentar mensagem de recuperação.
7. Caso contrário, apresentar mensagem de reprovação.

2. Contagem de Negativos e Soma dos Positivos

Entrada: valor real (`num`)

Processamento:

- armazenamento de 20 números reais em uma lista
- contagem de números negativos
- soma dos números positivos

Saída: quantidade de negativos e soma dos positivos

Algoritmo

1. Declarar e inicializar as variáveis `nums = [ ]` como lista, `nqty = 0`, `count = 0` do tipo inteiro e `num = 0.0`, `soma = 0.0` do tipo real.
2. Para `count` no intervalo de 1 até 20, executar:
 - i. Tentar ler um valor real e armazenar em `num`.
 - ii. Adicionar `num` ao final da lista `nums`.
 - iii. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
3. Para cada `num` em `nums`, executar:
 - i. Se `num` for menor que 0, incrementar `nqty` em 1.
 - ii. Senão, se `num` for maior que 0, somar `num` em `soma`.
4. Apresentar uma linha separadora.
5. Apresentar a quantidade de números negativos (`nqty`).
6. Apresentar a soma dos números positivos (`soma`).

3. Posição do Maior Valor na Lista

Entrada: valor inteiro (`num`)

Processamento:

- armazenamento de 10 números inteiros em uma lista
- identificação do maior valor e sua posição (sem uso de métodos prontos)

Saída: posição do maior valor

Algoritmo

1. Declarar e inicializar as variáveis `nums = [ ]` como lista, `count = 0`, `num = 0`, `pos = 0`, `index = 1` do tipo inteiro e `max_num = 0` do tipo inteiro.
2. Para `count` no intervalo de 1 até 10, executar:
 - i. Tentar ler um valor inteiro e armazenar em `num`.
 - ii. Adicionar `num` ao final da lista `nums`.
 - iii. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
3. Armazenar `nums[0]` em `max_num`.
4. Para `index` no intervalo de 1 até 9, executar:

- i. Se `nums[index]` for maior que `max_num`, armazenar `nums[index]` em `max_num` e `index` em `pos`.
5. Apresentar uma linha separadora.
6. Apresentar a posição do maior valor (`pos`).

4. Valores Maiores que 10 em Matriz 3x3

Entrada: valor inteiro (`num`)

Processamento:

- preenchimento de uma matriz 3x3
- contagem de números maiores que 10

Saída: quantidade de números maiores que 10

Algoritmo

1. Declarar e inicializar as variáveis `matrix = [ ]` como lista, `count_greater = 0` do tipo inteiro.
2. Para `i` no intervalo de 0 até 2, executar:
 - i. Declarar `row = [ ]` como lista.
 - ii. Para `j` no intervalo de 0 até 2, executar:
 - a. Tentar ler um valor inteiro e armazenar em `num`.
 - b. Adicionar `num` ao final da lista `row`.
 - c. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
 - iii. Adicionar `row` ao final da lista `matrix`.
3. Para cada `row` em `matrix`, executar:
 - i. Para cada `num` em `row`, executar:
 - a. Se `num` for maior que 10, incrementar `count_greater` em 1.
4. Apresentar uma linha separadora.
5. Apresentar a matriz formatada.
6. Apresentar a quantidade de números maiores que 10 (`count_greater`).

5. Soma dos Números Acima da Diagonal Principal

Entrada: valor inteiro (num)

Processamento:

- preenchimento de uma matriz 3x3
- soma dos elementos acima da diagonal principal

Saída: soma dos elementos acima da diagonal principal

Algoritmo

1. Declarar e inicializar as variáveis `matrix = [ ]` como lista, `soma = 0` do tipo inteiro.
2. Para `i` no intervalo de 0 até 2, executar:
 - i. Declarar `row = [ ]` como lista.
 - ii. Para `j` no intervalo de 0 até 2, executar:
 - a. Tentar ler um valor inteiro e armazenar em `num`.
 - b. Adicionar `num` ao final da lista `row`.
 - c. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
 - iii. Adicionar `row` ao final da lista `matrix`.
3. Para `i` no intervalo de 0 até 2, executar:
 - i. Para `j` no intervalo de 0 até 2, executar:
 - a. Se `j` for maior que `i`, somar `matrix[i][j]` em `soma`.
4. Apresentar uma linha separadora.
5. Apresentar a soma dos números acima da diagonal principal (`soma`).

6. Contagem de Pares e Ímpares

Entrada: valor inteiro (num)

Processamento:

- armazenamento de 20 números inteiros em uma lista
- contagem de números pares e ímpares

Saída: quantidade de pares e quantidade de ímpares

Algoritmo

1. Declarar e inicializar as variáveis `nums = [ ]` como lista, `count = 0`, `num = 0`, `even = 0`, `odd = 0` do tipo inteiro.
2. Para `count` no intervalo de 1 até 20, executar:
 - i. Tentar ler um valor inteiro e armazenar em `num`.
 - ii. Adicionar `num` ao final da lista `nums`.
 - iii. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
3. Para cada `num` em `nums`, executar:
 - i. Se `num` for par (`num % 2 == 0`), incrementar `even` em 1.
 - ii. Caso contrário, incrementar `odd` em 1.
4. Apresentar uma linha separadora.
5. Apresentar a quantidade de números pares (`even`).
6. Apresentar a quantidade de números ímpares (`odd`).

7. Média dos Cinco Primeiros e dos Cinco Últimos

Entrada: valor inteiro (`num`)

Processamento:

- armazenamento de 10 números inteiros em uma lista
- cálculo da média dos 5 primeiros elementos
- cálculo da média dos 5 últimos elementos

Saída: média dos 5 primeiros e média dos 5 últimos

Algoritmo

1. Declarar e inicializar as variáveis `nums = [ ]` como lista, `count = 0`, `num = 0` do tipo inteiro e `avg_one = 0.0`, `avg_two = 0.0` do tipo real.
2. Para `count` no intervalo de 1 até 10, executar:
 - i. Tentar ler um valor inteiro e armazenar em `num`.
 - ii. Adicionar `num` ao final da lista `nums`.
 - iii. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
3. Calcular a soma dos 5 primeiros: `sum_one = sum(nums[:5])`.

4. Calcular a soma dos 5 últimos: `sum_two = sum(nums[5:])`.
5. Calcular a média dos 5 primeiros: `avg_one = sum_one / 5`.
6. Calcular a média dos 5 últimos: `avg_two = sum_two / 5`.
7. Apresentar uma linha separadora.
8. Apresentar a média dos 5 primeiros (`avg_one`).
9. Apresentar a média dos 5 últimos (`avg_two`).

8. Procura de Valor em Lista Aleatória

Entrada: valor inteiro (`num`)

Processamento:

- geração de 20 números inteiros aleatórios entre 1 e 50
- verificação da existência de um valor informado na lista

Saída: resposta indicando se o valor existe e a lista completa

Algoritmo

1. Declarar e inicializar a variável `nums` como lista com 20 números inteiros aleatórios entre 1 e 50.
2. Declarar a variável `num = 0` do tipo inteiro.
3. Tentar ler um valor inteiro e armazenar em `num`.
4. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
5. Se `num` estiver presente na lista `nums`, apresentar mensagem de presença.
6. Caso contrário, apresentar mensagem de ausência.
7. Apresentar a lista completa (`nums`).

9. Soma por Faixas Numéricas

Entrada: valor inteiro (`num`)

Processamento:

- armazenamento de 10 números inteiros em uma lista
- soma dos números até 20 (inclusive)
- soma dos números entre 21 e 30 (inclusive)
- soma dos números maiores que 30

Saída: soma de cada faixa numérica

Algoritmo

1. Declarar e inicializar as variáveis `nums = [ ]` como lista, `count = 0`, `num = 0`, `first_batch = 0`, `second_batch = 0`, `third_batch = 0` do tipo inteiro.
2. Para `count` no intervalo de 1 até 10, executar:
 - i. Tentar ler um valor inteiro e armazenar em `num`.
 - ii. Adicionar `num` ao final da lista `nums`.
 - iii. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
3. Para cada `num` em `nums`, executar:
 - i. Se `num` for menor ou igual a 20, somar `num` em `first_batch`.
 - ii. Senão, se `num` for menor ou igual a 30, somar `num` em `second_batch`.
 - iii. Caso contrário, somar `num` em `third_batch`.
4. Apresentar uma linha separadora.
5. Apresentar a soma dos números até 20 (`first_batch`).
6. Apresentar a soma dos números entre 21 e 30 (`second_batch`).
7. Apresentar a soma dos números maiores que 30 (`third_batch`).

10. Média dos Números e Elementos Acima da Média

Entrada: valor real (`num`)

Processamento:

- armazenamento de 20 números reais em uma lista
- cálculo da média
- identificação dos elementos acima da média

Saída: média calculada e lista de elementos acima da média

Algoritmo

1. Declarar e inicializar as variáveis `nums = [ ]` como lista, `count = 0` do tipo inteiro e `num = 0.0`, `avg = 0.0`, `soma = 0.0` do tipo real.
2. Para `count` no intervalo de 1 até 20, executar:
 - i. Tentar ler um valor real e armazenar em `num`.

- ii. Adicionar num ao final da lista nums.
 - iii. Em caso de erro na leitura (valor inválido), apresentar mensagem de entrada inválida.
- 3. Para cada num em nums, somar num em soma.
- 4. Calcular a média: $avg = soma / 20$.
- 5. Apresentar uma linha separadora.
- 6. Apresentar a média (avg).
- 7. Apresentar uma linha separadora.
- 8. Para cada num em nums, executar:
 - i. Se num for maior que avg, apresentar o valor.